DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University

Submitted by:

ANJALI TOMAR
(2401010080)

B.Tech CSE (Core) SecA

Semester 4th

Submitted to
Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

Batch – 2024-2028

Lab Experiment 1

AIM: Design an Entity-Relationship (ER) diagram and convert it into relational tables for a College Database scenario.

COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

SOLUTION

I. **Identify Entities**: The core entities are **STUDENT**, **SEMSEC**, **SUBJECT**, and **IAMARKS**, represented using rectangles.

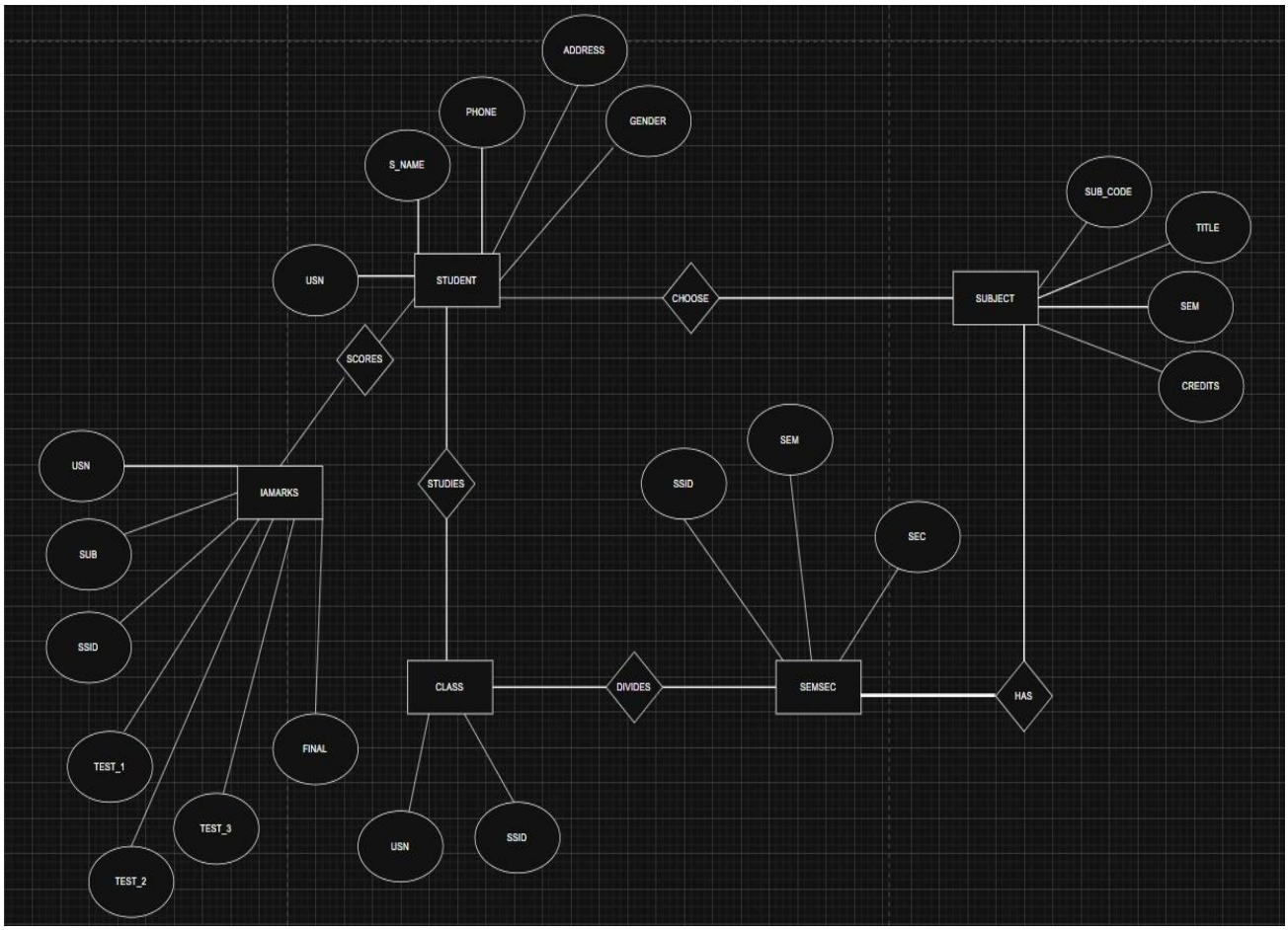
II. **Define Attributes:** Attributes are represented using ellipses. Primary keys are identified by underlining the attribute name.
STUDENT: USN (Primary Key), SName, Address, Phone No, Gender.
SEMSEC: SSID (Primary Key), Semester, Section.
SUBJECT: Subcode (Primary Key), Title, Semester, Credits.
IAMARKS: USN, Subcode, SSID, Test1, Test2, Test3, FinalIA.

III. **Multivalued Attributes**: **Phone No** is shown using a double ellipse, indicating a student may have multiple contact numbers.

IV. **Relationships:**

a) **CLASS**: A diamond-shaped relationship between STUDENT and SEMSEC with an **M:N** cardinality.
b) **IAMARKS (Associative Entity)**: Resolves the **M:N** relationship between STUDENT and SUBJECT.
c) **Cardinality**: Relationships between STUDENT-IAMARKS, SUBJECT-IAMARKS, and SEMSEC-IAMARKS are all **1:M**

V. **ER Diagram:**



Lab Experiment 2

AIM: Design an Entity-Relationship (ER) diagram and convert it into relational tables for a Company Database scenario.

COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Salary, Sex, SuperSSN, DNO)
DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)
PROJECT (PNo, PName, PLocation, DNO)
DLOCATION (DNo, DLoc)
WORKS_ON (SSN, PNo, Hours)
MANAGES (SSN, DNo)

SOLUTION

I. **Identify Entities**: The core entities include **EMPLOYEE**, **DEPARTMENT**, **PROJECT**, and **DLOCATION**.

II. **Define Attributes:** Attributes are represented using ellipses. Primary keys are identified by underlining the attribute name.

EMPLOYEE: SSN (Primary Key), Name, Address, Sex, Salary, SuperSSN, DNo.
DEPARTMENT: DNo (Primary Key), DName, MgrSSN, MgrStartDate.
PROJECT: PNo (Primary Key), PName, PLocation, DNo.
DLOCATION: DNo, DLoc.

III. **Primary Keys:** All primary keys are underlined to ensure data integrity.

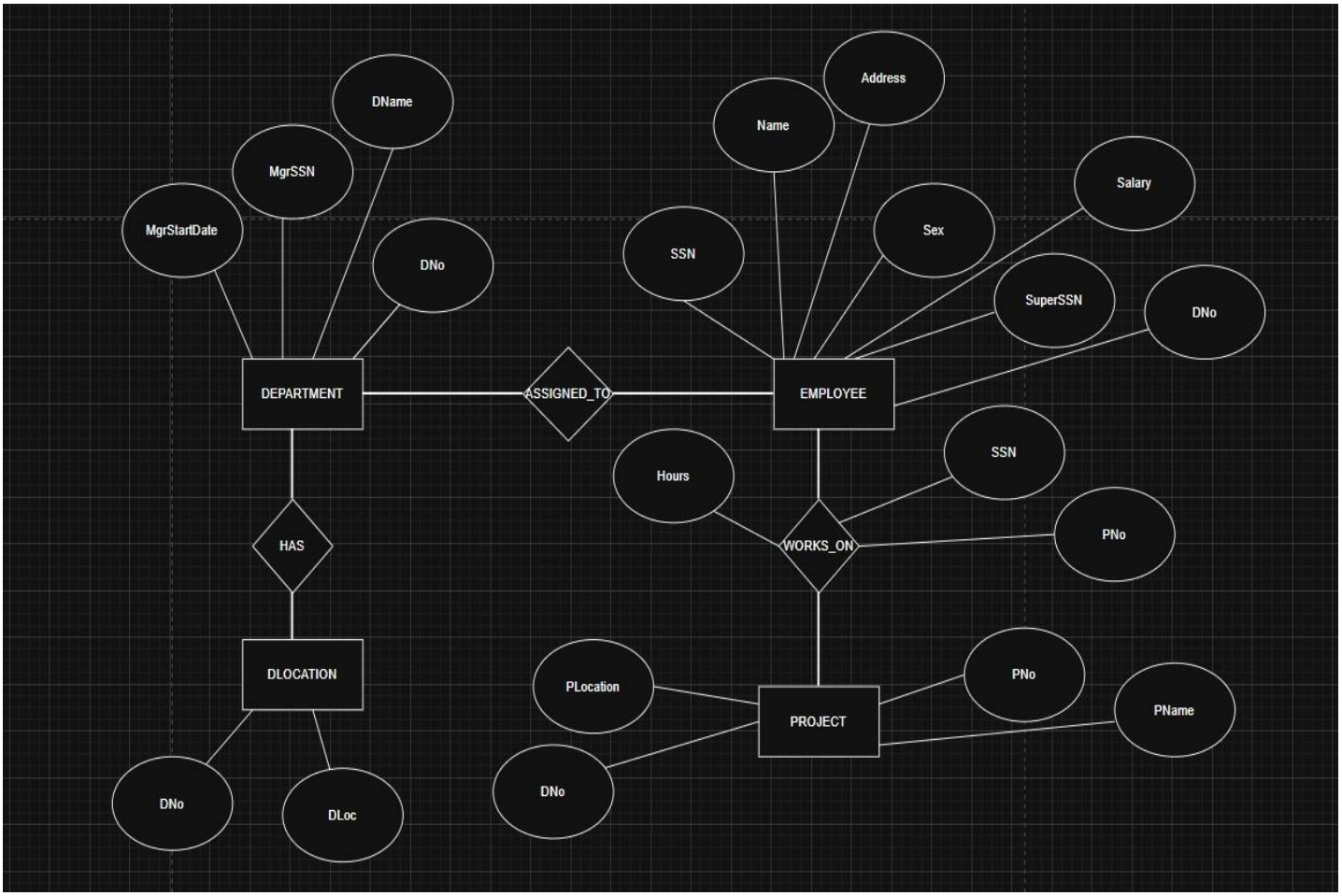
IV. **Relationships:**

a) **WORKS_ON:** An M:N relationship between EMPLOYEE and PROJECT, including the attribute Hours.

b) **MANAGES:** A 1:1 relationship between EMPLOYEE and DEPARTMENT.

c) **Departmental Connections: A 1:M relationship exists from DEPARTMENT to both PROJECT and DLOCATION.**

V. **ER Diagram:**



Lab Experiment 3

AIM: Database Schema for a customer-sale scenario:

Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following:

(a) Create tables with the appropriate integrity constant.

```sql
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

(b) Insert 10 records in each table.

```sql
INSERT INTO Customer VALUES
(1,'Amit'),
(2,'Neha'),
(3,'Rahul'),
(4,'Sneha'),
(5,'Karan'),
(6,'Priya'),
(7,'Ayesha'),
(8,'Rohit'),
(9,'Simran'),
(10,'Vikram');
```

```sql
INSERT INTO Item VALUES
(101,'Pen',50),
(102,'Book',150),
(103,'Bag',500),
(104,'Bottle',120),
(105,'Shoes',800),
(106,'Watch',1200),
(107,'Laptop',50000),
(108,'Mouse',300),
(109,'Keyboard',700),
(110,'Chair',1000);
```

```sql
INSERT INTO Sale VALUES
(1001,'2026-04-26',1,101,2),
(1002,'2026-04-26',2,102,1),
(1003,'2026-04-25',3,103,1),
(1004,'2026-04-26',4,104,3),
(1005,'2026-04-25',5,105,1),
(1006,'2026-04-26',1,106,1),
(1007,'2026-04-26',6,107,1),
(1008,'2026-04-25',7,108,2),
(1009,'2026-04-26',8,109,1),
(1010,'2026-04-26',9,110,1);
```

Result Grid | Filter Rows:

cust_id	cust_name
1	Amit
2	Neha
3	Rahul
4	Sneha
5	Karan
6	Priya
7	Ayesha
8	Rohit
9	Simran
10	Vikram

Result Grid | Filter Rows:

item_id	item_name	price
101	Pen	50
102	Book	150
103	Bag	500
104	Bottle	120
105	Shoes	800
106	Watch	1200
107	Laptop	50000
108	Mouse	300
109	Keyboard	700
110	Chair	1000

item 12

Result Grid | Filter Rows: | Edi

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-04-26	1	101	2
1002	2026-04-26	2	102	1
1003	2026-04-25	3	103	1
1004	2026-04-26	4	104	3
1005	2026-04-25	5	105	1
1006	2026-04-26	1	106	1
1007	2026-04-26	6	107	1
1008	2026-04-25	7	108	2
1009	2026-04-26	8	109	1
1010	2026-04-26	9	110	1

sale 13

(c) List bills for current date with customer names and item numbers.

Limit to 1000 rows

```
SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_name	item_id
1001	2026-04-26	Amit	101
1002	2026-04-26	Neha	102
1004	2026-04-26	Sneha	104
1006	2026-04-26	Amit	106
1007	2026-04-26	Priya	107
1009	2026-04-26	Rohit	109
1010	2026-04-26	Simran	110

Result Grid | Form Editor | Field Types

(d) List total Bill details with qty, price, and final amount.

```
        c.cust_name,
        i.item_name,
        s.qty_sold,
        i.price,
        (s.qty_sold * i.price) AS final_amount
    FROM Sale s
    JOIN Customer c ON s.cust_id = c.cust_id
    JOIN Item i ON s.item_id = i.item_id;
```

Result Grid

bill_no	cust_name	item_name	qty_sold	price	final_amount
1001	Amit	Pen	2	50	100
1002	Neha	Book	1	150	150
1003	Rahul	Bag	1	500	500
1004	Sneha	Bottle	3	120	360
1005	Karan	Shoes	1	800	800
1006	Amit	Watch	1	1200	1200
1007	Priya	Laptop	1	50000	50000
1008	Ayesha	Mouse	2	300	600
1009	Rohit	Keyboard	1	700	700
1010	Simran	Chair	1	1000	1000

(e) List details of customers who bought products with price > 200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid

cust_id	cust_name
3	Rahul
5	Karan
1	Amit
6	Priya
7	Ayesha
8	Rohit
9	Simran

(f) Count of products bought by each customer.

```
SELECT c.cust_id, c.cust_name, SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

cust_id	cust_name	total_products
1	Amit	3
2	Neha	1
3	Rahul	1
4	Sneha	3
5	Karan	1
6	Priya	1
7	Ayesha	2
8	Rohit	1
9	Simran	1

(g) Products bought by customer with cust_id 5.

```
SELECT c.cust_name, i.item_name, s.qty_sold
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE c.cust_id = 5;
```

cust_name	item_name	qty_sold
Karan	Shoes	1

(h) List item details sold as of today.

```sql
SELECT DISTINCT i.item_id, i.item_name, i.price
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid

item_id	item_name	price
101	Pen	50
102	Book	150
104	Bottle	120
106	Watch	1200
107	Laptop	50000
109	Keyboard	700
110	Chair	1000

(i) Create view for bill details.

```sql
CREATE VIEW Bill_Details AS
SELECT
    s.bill_no,
    s.bill_date,
    c.cust_name,
    i.item_name,
    s.qty_sold,
    (s.qty_sold * i.price) AS total_amount
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;
```

(j) Create view for weekly daily sales.

```sql
CREATE VIEW Daily_Sales AS
SELECT
    s.bill_date,
    SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
GROUP BY s.bill_date;
```

Lab Experiment 4

AIM: Consider the following table: Table: CLASS (ID,Name)

```
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;
SET SQL_SAFE_UPDATES = 0;

DROP TABLE IF EXISTS student_class;
CREATE TABLE student_class (
    roll_id INT,
    student_name VARCHAR(20)
);

INSERT INTO student_class VALUES (1, 'rohit'), (2, 'sara'), (4, 'nikhil');
```

roll_id	student_name
1	rohit
2	sara
4	nikhil

(a) INSERT INTO class VALUES (5,mahkar');.

```
SET autocommit = 0;
START TRANSACTION;
INSERT INTO student_class VALUES (5, 'mahkar');
COMMIT;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	mahin

(b) Update student_name with Savepoints A,B, and C

```
START TRANSACTION;
UPDATE student_class SET student_name = 'abhishek' WHERE roll_id = 5;
SAVEPOINT pt_a;

INSERT INTO student_class VALUES (6, 'tanya'); SAVEPOINT
pt_b;

INSERT INTO student_class VALUES (7, 'rohit');
SAVEPOINT pt_c;
```

Select 1: Shows all 5 records

```
SELECT * FROM student_class;
ROLLBACK TO pt_b;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek
6	tanya
7	rohit

Select 2: id 7 is gone; id 6 remains

```
SELECT * FROM student_class;
ROLLBACK TO pt_a;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek
6	tanya

Select 3: id 6 is gone; id 5 remains as 'abhishek'

```
SELECT * FROM student_class;
COMMIT;
```

roll_id	student_name
1	rohit
2	sara
4	nikhil
5	abhishek

Lab Experiment 4.2

AIM: Consider the following table and perform the following TCL-based SQL script:

acc_id	holder_name	curr_balance
101	ishan	5000
102	megha	7000
103	daksh	6000

a) Insert data into the table: (104, ‘kavya’, 4000)

```
START TRANSACTION;
INSERT INTO bank_ledger VALUES (104, 'kavya', 4000);
SET autocommit = 0;
SAVEPOINT s1;
```

acc_id	holder_name	curr_balance
101	ishan	5000
102	megha	7000
103	daksh	6000
104	kavya	4000

b) Update table, set balance where acc_id = 101 and acc_id = 102

```
UPDATE bank_ledger
SET curr_balance = curr_balance + 1500 WHERE acc_id = 101;
UPDATE bank_ledger
SET curr_balance = curr_balance - 1000 WHERE acc_id = 102;
SAVEPOINT s2;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000

c) Delete from table, where acc_id = 103 and insert ‘rehan’ at acc_id = 105

```
DELETE FROM bank_ledger WHERE acc_id = 103;
INSERT INTO bank_ledger VALUES (105, 'rehan', 3000);
SAVEPOINT s3;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
104	kavya	4000
105	rehan	3000

d) Undoes the delete of daksh, the insert of rehan, and name change of megha.

```
ROLLBACK TO s2;
SELECT * FROM bank_ledger;
COMMIT;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000

e) Insert VALUES(106, ‘zoya’, 4500) and SAVEPOINT s4.

```
INSERT INTO bank_ledger VALUES
(106, 'zoya', 4500);
SAVEPOINT s4;
```

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000
106	zoya	4500

f) Update table, set balance where acc_id = 101 and 104.

```
UPDATE bank_ledger SET curr_balance
= curr_balance + 500
WHERE acc_id IN (101, 104);
SELECT * FROM bank_ledger;
```

acc_id	holder_name	curr_balance
101	ishan	7000
102	megha	6000
103	daksh	6000
104	kavya	4500
106	zoya	4500

acc_id	holder_name	curr_balance
101	ishan	6500
102	megha	6000
103	daksh	6000
104	kavya	4000
106	zoya	4500

g) ROLLBACK to s4 and commit.

```
ROLLBACK TO s4;
SELECT * FROM bank_ledger;
COMMIT;
```

Lab Experiment 5

ASSIGNMENT-5

Name: Anjali Tomar Roll NO.: 2401010080

(Exercise on retrieving records from the table)

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

(a) Find out the employee id, names, salaries of all the employees

(b) List out the employees who works under manager 100

(c) Find the names of the employees who have a salary greater than or equal to 4800

(d) List out the employees whose last name is 'AUSTIN'

(e) Find the names of the employees who works in departments 60,70 and 80

(f) Display the unique Manager_Id.

Solution-

>TABLE CREATION -

```sql
USE 5Anjali080;
CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(100),
    Phone_Number VARCHAR(15),
    Hire_Date DATE,
    Job_Id VARCHAR(20),
    Salary INT,
    Commission_Act FLOAT,
    Manager_Id INT,
    Department_Id INT
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:36:04	CREATE DATABASE 5Anjali080	1 row(s) affected	0.032 sec
2	22:37:45	USE 5Anjali080	0 row(s) affected	0.000 sec
3	22:37:55	USE 5Anjali080	0 row(s) affected	0.000 sec
4	22:37:55	CREATE TABLE EMPLOYEES (Employee_Id INT PRIMARY KEY, First_Name VARCHAR(50), Last_Na...	0 row(s) affected	0.062 sec

>DATA INSERTION -

```
.com', '9876543210', '2020-01-10', 'IT_PROG', 5000, NULL, 100, 60)
com', '9876543211', '2021-03-15', 'HR_REP', 4500, NULL, 101, 70),
l.com', '9876543212', '2019-07-20', 'IT_PROG', 6000, NULL, 100, 60
.com', '9876543213', '2022-05-25', 'SALES', 4800, 0.1, 102, 80),
l.com', '9876543214', '2018-11-30', 'SALES', 5200, 0.2, 100, 80),
il.com', '9876543215', '2020-06-18', 'HR_REP', 4700, NULL, 101, 70
il.com', '9876543216', '2017-09-09', 'IT_PROG', 5500, NULL, 100, 6
il.com', '9876543217', '2021-12-01', 'SALES', 4900, 0.15, 102, 80)
l.com', '9876543218', '2019-04-14', 'IT_PROG', 5100, NULL, 100, 60
il.com', '9876543219', '2022-02-20', 'HR_REP', 4300, NULL, 101, 70
il.com', '9876543220', '2016-08-11', 'SALES', 7000, 0.25, 102, 80)
ail.com', '9876543221', '2020-10-05', 'IT_PROG', 4800, NULL, 100,
gmail.com', '9876543222', '2018-01-17', 'HR_REP', 4600, NULL, 101,
il.com', '9876543223', '2021-07-23', 'SALES', 5300, 0.18, 102, 80)
gmail.com', '9876543224', '2019-03-30', 'IT_PROG', 6200, NULL, 100
```

(a) Find employee id, names, and salaries of all employees

```
USE 5Anjali080;
SELECT Employee_Id, First_Name, Last_Name, Salary
FROM EMPLOYEES;
```

Result Grid

Employee_Id	First_Name	Last_Name	Salary
1	Amit	Sharma	5000
2	Riya	Verma	4500
3	Karan	Mehta	6000
4	Neha	AUSTIN	4800
5	Rahul	Singh	5200
6	Priya	Kapoor	4700
7	Ankit	AUSTIN	5500
8	Simran	Kaur	4900
9	Vikas	Gupta	5100
10	Pooja	Yadav	4300

EMPLOYEES 1

(b) List employees who work under manager 100

```
USE 5Anjali080;
SELECT *
FROM EMPLOYEES
WHERE Manager_Id = 100;
```

Result Grid | Filter Rows: | Edit: | Export/Import

Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_
1	Amit	Sharma	amit@gmail.com	9876543210	2020-
3	Karan	Mehta	karan@gmail.com	9876543212	2019-
5	Rahul	Singh	rahul@gmail.com	9876543214	2018-
7	Ankit	AUSTIN	ankit@gmail.com	9876543216	2017-
9	Vikas	Gupta	vikas@gmail.com	9876543218	2019-
12	Sneha	AUSTIN	sneha@gmail.com	9876543221	2020-
15	Manish	Chopra	manish@gmail.com	9876543224	2019-
NULL	NULL	NULL	NULL	NULL	NULL

(c) Employees with salary ≥ 4800

```
USE 5Anjali080;
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Salary >= 4800;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

First_Name	Last_Name
Amit	Sharma
Karan	Mehta
Neha	AUSTIN
Rahul	Singh
Ankit	AUSTIN
Simran	Kaur
Vikas	Gupta
Arjun	Patel
Sneha	AUSTIN
Kavya	Reddy

EMPLOYEES 4 | Read Only

(d) Employees whose last name is 'AUSTIN'

```
USE 5Anjali080;
SELECT *
FROM EMPLOYEES
WHERE Last_Name = 'AUSTIN';
```

Result Grid

Employee_Id	First_Name	Last_Name	Email	Phone_Number	Hire_D
4	Neha	AUSTIN	neha@gmail.com	9876543213	2022-0
7	Ankit	AUSTIN	ankit@gmail.com	9876543216	2017-0
12	Sneha	AUSTIN	sneha@gmail.com	9876543221	2020-1
NULL	NULL	NULL	NULL	NULL	NULL

(e) Employees working in departments 60, 70, 80

```
USE 5Anjali080;
SELECT First_Name, Last_Name
FROM EMPLOYEES
WHERE Department_Id IN (60, 70, 80);
```

Result Grid

First_Name	Last_Name
Amit	Sharma
Riya	Verma
Karan	Mehta
Neha	AUSTIN
Rahul	Singh
Priya	Kapoor
Ankit	AUSTIN
Simran	Kaur
Vikas	Gupta
Pooja	Yadav

(f) Display unique Manager_Id

```
USE 5Anjali080;
SELECT DISTINCT Manager_Id
FROM EMPLOYEES;
```

Result Grid

Manager_Id
100
101
102

Lab Experiment – 6

ASSIGNMENT-6

Name: Anjali Tomar Roll NO.:2401010080

(Exercise on updating records in table)

Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due) (a) Insert five records

(b) Find the names of clients whose bal_due> 5000 .

(c) Change the bal_due of ClientNO " C123" to Rs. 5100 (d) Change the name of Client_master to Client12.

(e) Display the bal_due heading as "BALANCE"

SOLUTION-

>TABLE CREATION-

```sql
USE 6Anjali080;
CREATE TABLE Client_master (
    ClientNO VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    bal_due INT
);
```

(a) Insert 5 records

```sql
USE 6Anjali080;
INSERT INTO Client_master VALUES
('C101', 'Amit Sharma', 'Street 1', 'Delhi', 'Delhi', 4000),
('C102', 'Riya Verma', 'Street 2', 'Mumbai', 'Maharashtra', 6000),
('C123', 'Karan Mehta', 'Street 3', 'Pune', 'Maharashtra', 4500),
('C104', 'Neha Singh', 'Street 4', 'Chennai', 'Tamil Nadu', 7000),
('C105', 'Rahul Gupta', 'Street 5', 'Kolkata', 'West Bengal', 3000
```

(b) Names of clients whose bal_due > 5000

```
USE 6Anjali080;
SELECT Name
FROM Client_master
WHERE bal_due > 5000;
```

Result Grid

Name
Riya Verma
Neha Singh

(c) Change bal_due of ClientNO 'C123' to 5100

```
USE 6Anjali080;
UPDATE Client_master
SET bal_due = 5100
WHERE ClientNO = 'C123';
```

(d) Change table name to Client12

```
USE 6Anjali080;
RENAME TABLE Client_master TO Client12;
```

(e) Display bal_due heading as "BALANCE"

```
USE 6Anjali080;
SELECT bal_due AS BALANCE
FROM Client12;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BALANCE
4000
6000
7000
3000
5100

Result Grid
Form Editor
Field Types

Lab Experiment 7

Objective: Rollback and Commit commands Create Teacher table with the following fields (Name,
DeptNo, Date of joining, DeptName, Location, Salary)

a) Insert five records.
b) Give Increment of 25% salary for Mathematics Department.
c) Perform Rollback command.
d) Give increment of 15% salary for Commerce Department.
e) Perform Commit command.

TABLE CREATION:

```
CREATE TABLE Teacher (
    Name VARCHAR(50),
    DeptNo INT,
    Date_of_Joining DATE,
    DeptName VARCHAR(50),
    Location VARCHAR(50),
    Salary INT
);
```

49	19:51:50	CREATE TABLE Teacher (	Name VARCHAR(50),	DeptNo INT,	Date_of_Joining DATE,	DeptName ...	0 row(s) affected
50	19:52:16	CREATE TABLE Teacher (	Name VARCHAR(50),	DeptNo INT,	Date_of_Joining DATE,	DeptName ...	Error Code: 1050. Table 'teacher' already exists
51	19:53:46	DROP TABLE Teacher					0 row(s) affected
52	19:53:59	CREATE TABLE Teacher (	Name VARCHAR(50),	DeptNo INT,	Date_of_Joining DATE,	DeptName ...	0 row(s) affected

(a) Insert five records.

```
INSERT INTO Teacher VALUES
('Amit', 101, '2020-06-15', 'Mathematics', 'Delhi', 50000),
('Neha', 102, '2019-08-10', 'Commerce', 'Mumbai', 45000),
('Ravi', 103, '2021-01-20', 'Mathematics', 'Delhi', 52000),
('Priya', 104, '2018-03-12', 'Science', 'Pune', 55000),
('Karan', 105, '2022-07-05', 'Commerce', 'Kolkata', 40000);
```

(b) Give Increment of 25% salary for Mathematics Department

```
START TRANSACTION;

UPDATE Teacher
SET Salary = Salary + (Salary * 0.25)
WHERE DeptName = 'Mathematics';
```

(c) Perform Rollback command.

```
ROLLBACK;
SELECT * FROM teacher WHERE DeptName = 'mathematics';
```

Result Grid

Name	DeptNo	Date_of_joining	DeptName	Location	Salary
Amit	101	2020-06-15	Mathematics	Delhi	50000
Ravi	103	2021-01-20	Mathematics	Delhi	52000

(d) Give increment of 15% salary for Commerce Department.

```
UPDATE Teacher
SET Salary = Salary + (Salary * 0.15)
WHERE DeptName = 'Commerce';
```

(e) Perform Commit command.

```
COMMIT;
SELECT * FROM teacher;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Name	DeptNo	Date_of_joining	DeptName	Location	Salary
Amit	101	2020-06-15	Mathematics	Delhi	50000
Neha	102	2019-08-10	Commerce	Mumbai	51750
Ravi	103	2021-01-20	Mathematics	Delhi	52000
Priya	104	2018-03-12	Science	Pune	55000
Karan	105	2022-07-05	Commerce	Kolkata	46000

Lab Experiment 8

Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB) a) Insert five records.

b) Calculate total salesamount in each branch.
c) Calculate average salesamount in each branch.
d) Display all the salesmen, DOB who are born in the month of December as day incharacter format i.e. 21-Dec-09.
e) Display the name and DOB of salesman in alphabetical order of the month.

```
CREATE TABLE Sales (
    SalesNo INT PRIMARY KEY,
    Salesname VARCHAR(50),
    Branch VARCHAR(50),
    Salesamount INT,
    DOB DATE
);
```

59	20:04:14	CREATE DATABASE assignment8	1 row(s) affected
60	20:04:20	USE assignment8	0 row(s) affected
61	20:04:34	CREATE TABLE Sales (SalesNo INT PRIMARY KEY, Salesname VARCHAR(50), Branch VARCHAR(...	0 row(s) affected

(a) Insert five records.

```
INSERT INTO Sales VALUES
(1, 'Aman', 'Delhi', 5000, '2002-12-21'),
(2, 'Anjali', 'Mumbai', 7000, '2001-05-15'),
(3, 'Akash', 'Delhi', 6000, '2003-12-10'),
(4, 'Vicky', 'Mumbai', 8000, '2000-11-25'),
(5, 'Kanika', 'Chennai', 5500, '2002-12-05');
```

(b) Calculate total salesamount in each branch.

SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12*

Limit to 1000 rows

```sql
SELECT Branch, SUM(Salesamount) AS Total_Sales
FROM Sales
GROUP BY Branch;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Branch	Total_Sales
Delhi	11000
Mumbai	15000
Chennai	5500

Result Grid

Form

(c) Calculate average salesamount in each branch.

Limit to 1000 rows

```sql
SELECT Branch, AVG(Salesamount) AS Avg_Sales
FROM Sales
GROUP BY Branch;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Branch	Avg_Sales
Delhi	5500.0000
Mumbai	7500.0000
Chennai	5500.0000

Result Grid

Form Editor

(d) Display all the salesmen, DOB who are born in the month of December as day in characterformat i.e. 21-Dec-09.

```
SELECT Salesname, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
FROM Sales
WHERE MONTH(DOB) = 12;
```

Result Grid

Salesname	DOB
Aman	21-Dec-02
Akash	10-Dec-03
Kanika	05-Dec-02

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
SELECT Salesname, DOB
FROM Sales
ORDER BY MONTHNAME(DOB);
```

Result Grid

Salesname	DOB
Aman	2002-12-21
Akash	2003-12-10
Kanika	2002-12-05
Anjali	2001-05-15
Vicky	2000-11-25

Output

Action Output

#	Time	Action	Message
61	20:04:34	CREATE TABLE Sales (SalesNo INT PRIMARY KEY, Salesname VARCHAR(50), Branch VARCHAR(...	0 row(s) affected
62	20:08:03	INSERT INTO Sales VALUES (1, 'Aman', 'Delhi', 5000, '2002-12-21'), (2, 'Anjali', 'Mumbai', 7000, '2001-05-15'),...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
63	20:09:17	SELECT Branch, SUM(Salesamount) AS Total_Sales FROM Sales GROUP BY Branch LIMIT 0, 1000	3 row(s) returned
64	20:09:51	SELECT Branch, AVG(Salesamount) AS Avg_Sales FROM Sales GROUP BY Branch LIMIT 0, 1000	3 row(s) returned
65	20:10:29	SELECT Salesname, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB FROM Sales WHERE MONTH(DOB) = 12 ...	3 row(s) returned
66	20:11:09	SELECT Salesname, DOB FROM Sales ORDER BY MONTHNAME(DOB) LIMIT 0, 1000	5 row(s) returned

Lab Experiment 9

Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas are as follows:

Departments (dept no , dept name , dept_location);
Employees (emp_id , emp_name , emp_salary,dept_no);

a) Develop a query to grant all privileges of employees table into departments table.
b) Develop a query to grant some privileges of employees table into departments table.
c) Develop a query to revoke all privileges of employees table from departments table.
d) Develop query to revoke some privileges of employees table from departments table.
e) Write a query to implement the save point.

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
```

```
CREATE TABLE Departments (
    dept_no INT PRIMARY KEY,
    dept_name VARCHAR(50),
    dept_location VARCHAR(50)
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    emp_salary INT,
    dept_no INT
);
```

(a) Develop a query to grant all privileges of employees table into departments table.

```
CREATE USER 'user1'@'localhost' IDENTIFIED BY '1234';

GRANT ALL PRIVILEGES
ON Employees
TO 'user1'@'localhost';
```

72	20:17:11	CREATE USER 'user1'@'localhost' IDENTIFIED BY '1234'	0 row(s) affected
73	20:17:11	GRANT ALL PRIVILEGES ON Employees TO 'user1'@'localhost'	0 row(s) affected

(b) Develop a query to grant some privileges of employees table into departments table.

```
GRANT SELECT, INSERT
ON Employees
TO 'user1'@'localhost';
```

(c) Develop a query to revoke all privileges of employees table from departments table.

```
REVOKE ALL PRIVILEGES
ON Employees
FROM 'user1'@'localhost';
```

(d) Develop a query to revoke some privileges of employees table from departments table.

```

REVOKE SELECT, INSERT
ON Employees
FROM 'user1'@'localhost';
```

(e) Write a query to implement the save point.

SQL File 7* | SQL File 8* | SQL File 9* | SQL File 10* | SQL File 11* | SQL File 12*

Limit to 1000 rows

```sql
START TRANSACTION;

INSERT INTO Employees VALUES (1, 'Aman', 5000, 101);

SAVEPOINT sp1;

INSERT INTO Employees VALUES (2, 'Riya', 6000, 102);

ROLLBACK TO sp1;

COMMIT;
```

Output

Action Output

#	Time	Action	Message
80	20:20:43	START TRANSACTION	0 row(s) affected
81	20:20:43	INSERT INTO Employees VALUES (1, 'Aman', 5000, 101)	1 row(s) affected
82	20:20:43	SAVEPOINT sp1	0 row(s) affected
83	20:20:43	INSERT INTO Employees VALUES (2, 'Riya', 6000, 102)	1 row(s) affected
84	20:20:43	ROLLBACK TO sp1	0 row(s) affected
85	20:20:43	COMMIT	0 row(s) affected

Lab Experiment 10

Using the tables "DEPARTMENTS" and "EMPLOYEES" from Lab 9 perform the following queries:

a) Display the employee details, departments that the departments are same in both theemp and dept.
b) Display the employee name and Department name by implementing a left outer join.
c) Display the employee name and Department name by implementing a right outer join.
d) Display the details of those who draw the salary greater than the average salary.

```sql
CREATE DATABASE assignment10;
USE assignment10;
```

```sql
CREATE TABLE Departments (
    dept_no INT PRIMARY KEY,
    dept_name VARCHAR(50),
    dept_location VARCHAR(50)
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    emp_salary INT,
    dept_no INT
);
```

```sql
INSERT INTO Departments VALUES
(101, 'HR', 'Delhi'),
(102, 'IT', 'Mumbai'),
(103, 'Sales', 'Chennai');

INSERT INTO Employees VALUES
(1, 'Chavi', 5000, 101),
(2, 'Vanshika', 7000, 102),
(3, 'Anjali', 6000, 101),
(4, 'Kanika', 8000, 104);
```

(a) Display the employee details, departments that the departments are same in both the empand dept.

```sql
SELECT e.*, d.dept_name, d.dept_location
FROM Employees e
INNER JOIN Departments d
ON e.dept_no = d.dept_no;
```

Result Grid

emp_id	emp_name	emp_salary	dept_no	dept_name	dept_location
1	Chavi	5000	101	HR	Delhi
2	Vanshika	7000	102	IT	Mumbai
3	Anjali	6000	101	HR	Delhi

(b) Display the employee name and Department name by implementing a left outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_no = d.dept_no;
```

Result Grid

emp_name	dept_name
Chavi	HR
Vanshika	IT
Anjali	HR
Kanika	NULL

(c) Display the employee name and Department name by implementing a right outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_no = d.dept_no;
```

Result Grid

emp_name	dept_name
Anjali	HR
Chavi	HR
Vanshika	IT
NULL	Sales

(d) Display the details of those who draw the salary greater than the average salary.

Limit to 1000 rows

```sql
SELECT emp_name, emp_salary
FROM Employees
WHERE emp_salary > (
    SELECT AVG(emp_salary) FROM Employees
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	emp_salary
Vanshika	7000
Kanika	8000

Result Grid

Form Editor

Output

Action Output

#	Time	Action	Message
91	20:24:20	INSERT INTO Departments VALUES (101, 'HR', 'Delhi'), (102, 'IT', 'Mumbai'), (103, 'Sales', 'Chennai')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
92	20:24:20	INSERT INTO Employees VALUES (1, 'Chavi', 5000, 101), (2, 'Vanshika', 7000, 102), (3, 'Anjali', 6000, 101), (4...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
93	20:25:33	SELECT e.*, d.dept_name, d.dept_location FROM Employees e INNER JOIN Departments d ON e.dept_no = ...	3 row(s) returned
94	20:26:58	SELECT e.emp_name, d.dept_name FROM Employees e LEFT JOIN Departments d ON e.dept_no = d.dept_n...	4 row(s) returned
95	20:27:45	SELECT e.emp_name, d.dept_name FROM Employees e RIGHT JOIN Departments d ON e.dept_no = d.dept...	4 row(s) returned
96	20:28:23	SELECT emp_name, emp_salary FROM Employees WHERE emp_salary > (SELECT AVG(emp_salary) FR...	2 row(s) returned